

```

1  /**
2   * control-edicion.js
3   * Sistema de edición visual rápida para la Alcaldía Local Municipal
4   */
5
6  document.addEventListener('DOMContentLoaded', () => {
7      const params = new URLSearchParams(window.location.search);
8      const esModoEdicion = params.get('modo') === 'editar';
9
10     if (!esModoEdicion) {
11         // Congelar el sitio: desactivar edición de texto
12         const editables = document.querySelectorAll('[contenteditable="true"]');
13         editables.forEach(el => {
14             el.setAttribute('contenteditable', 'false');
15         });
16
17         // Congelar el sitio: remover interacción de cambio de imagen
18         const imagenesEditables = document.querySelectorAll(
19             'img[onclick="cambiarImagenVisual"]');
20         imagenesEditables.forEach(img => {
21             img.removeAttribute('onclick');
22             img.style.cursor = 'default';
23         });
24     });
25
26     /**
27     * Activa el clic del input de archivo oculto
28     * @param {string} idImg - ID de la imagen a cambiar
29     * @param {string} idInput - ID del input file oculto
30     */
31     function cambiarImagenVisual(idImg, idInput) {
32         const input = document.getElementById(idInput);
33         if (input) {
34             input.click();
35         }
36     }
37
38     /**
39     * Previsualiza la imagen cargada localmente
40     * @param {HTMLInputElement} input - El input file que cambió
41     * @param {string} idImg - ID del elemento img a actualizar
42     */
43     function previsualizarNuevaImagen(input, idImg) {
44         if (input.files && input.files[0]) {
45             const reader = new FileReader();
46             reader.onload = function(e) {
47                 const img = document.getElementById(idImg);
48                 if (img) {
49                     img.src = e.target.result;
50                 }
51             };
52             reader.readAsDataURL(input.files[0]);
53         }
54     }
55     // =====
56     // CONTROL DE EDICIÓN VISUAL HÍBRIDO - ALCALDÍA LOCAL MUNICIPAL (PORTÁTIL)
57     // =====
58
59     const urlParams = new URLSearchParams(window.location.search);
60     const modoEditarActivo = urlParams.get('modo') === 'editar';
61
62     // SI EL USUARIO NO ENTRÓ CON LA URL SECRETA, CONGELAMOS EL SITIO INMEDIATAMENTE
63     if (!modoEditarActivo) {
64         document.querySelectorAll('[contenteditable="true"]').forEach(elemento => {
65             elemento.setAttribute('contenteditable', 'false');
66         });

```

```

67
68     document.querySelectorAll('img[onclick]').forEach(imagen => {
69         imagen.removeAttribute('onclick');
70         imagen.style.cursor = 'default';
71     });
72 } else {
73     // =====
74     // MODO EDICIÓN ACTIVO: INYECTAR BOTONES DE CONTROL DUAL
75     // =====
76
77     const contenedorBotonesHTML = `
78         <div id="panel-guardado-local" style="position:fixed; bottom:30px; left:30px;
79         display:flex; gap:12px; z-index:1000; font-family:Arial, sans-serif;">
80             <button id="btn-sincronizar-fast" style="padding:12px 20px;
81             background:#28a745; color:white; border:none; border-radius:5px;
82             font-weight:bold; cursor:pointer; box-shadow:0 4px 10px rgba(0,0,0,0.2);
83             transition: background 0.2s;">
84                 🔄 Sincronizar Tilde/Cambio (Python)
85             </button>
86             <button id="btn-descargar-backup" style="padding:12px 20px;
87             background:#007bff; color:white; border:none; border-radius:5px;
88             font-weight:bold; cursor:pointer; box-shadow:0 4px 10px rgba(0,0,0,0.2);
89             transition: background 0.2s;">
90                 📄 Descargar HTML Completo (Respaldo)
91             </button>
92         </div>
93     `;
94     document.body.insertAdjacentHTML('beforeend', contenedorBotonesHTML);
95
96     const nombreArchivo = window.location.pathname.split("/").pop() || "index.html";
97
98     // =====
99     // FUNCIÓN INTERNA GLOBAL DE PURIFICACIÓN (LIMPIEZA DE ATRIBUTOS PARA EL PÚBLICO)
100    // =====
101    function obtenerHTMLPurificado() {
102        // Clonamos el documento actual en la memoria RAM para no alterar lo que el
103        // usuario ve en pantalla
104        const clonDocumento = document.documentElement.cloneNode(true);
105
106        // Ocultamos el panel de botones dentro del clon para que no se grave a sí mismo
107        // en el archivo final
108        const panelEnClon = clonDocumento.querySelector('#panel-guardado-local');
109        if (panelEnClon) panelEnClon.remove();
110
111        // Congelamos todos los textos editables pasándolos a false de fábrica en el
112        // archivo resultante
113        clonDocumento.querySelectorAll('[contenteditable="true"]').forEach(elemento => {
114            elemento.setAttribute('contenteditable', 'false');
115        });
116
117        // Congelamos las imágenes informativas eliminando el disparador del cargador
118        // local
119        clonDocumento.querySelectorAll('img[onclick]').forEach(imagen => {
120            imagen.removeAttribute('onclick');
121            imagen.style.cursor = 'default';
122        });
123
124        // Retornamos el código limpio estructurado con su respectivo Doctype
125        return "<!DOCTYPE html>\n" + clonDocumento.outerHTML;
126    }
127
128    // =====
129    // LÓGICA BOTÓN VERDE: SINCRONIZACIÓN AUTOMÁTICA VELOZ (REQUERIRÁ PYTHON)
130    // =====
131    document.getElementById('btn-sincronizar-fast').addEventListener('click', async () =>
132    {
133        const btnSincro = document.getElementById('btn-sincronizar-fast');

```

```

122     const panelGlobal = document.getElementById('panel-guardado-local');
123
124     btnSincro.innerText = "Sincronizando...";
125     btnSincro.disabled = true;
126
127     // Ocultamos el panel en pantalla un milisegundo por seguridad antes del proceso
128     panelGlobal.style.display = 'none';
129     const codigoLimpio = obtenerHTMLPurificado();
130     panelGlobal.style.display = 'flex'; // Restaurado inmediatamente
131
132     try {
133         // Envía la actualización purificada a la API del servidor local de Python
134         // en el puerto 8000
135         const respuesta = await fetch('http://localhost:8000/api/guardar-html', {
136             method: 'POST',
137             headers: { 'Content-Type': 'application/json' },
138             body: JSON.stringify({
139                 archivo: nombreArchivo,
140                 html: codigoLimpio
141             })
142         });
143
144         if (respuesta.ok) {
145             btnSincro.innerText = "✅ ;Sincronizado!";
146             setTimeout(() => { btnSincro.innerText = "🔄 Sincronizar Tilde/Cambio (Python)"; }, 2000);
147         } else {
148             throw new Error();
149         }
150     } catch (error) {
151         alert("No se pudo sincronizar de forma rápida. Para usar este botón, necesitas tener el servicio de Python corriendo en el puerto 8000. Mientras tanto, puedes usar el botón azul de respaldo.");
152         btnSincro.innerText = "🔄 Sincronizar Tilde/Cambio (Python)";
153     } finally {
154         btnSincro.disabled = false;
155     }
156 });
157
158 // =====
159 // LÓGICA BOTÓN AZUL: DESCARGA DIRECTA RADICAL (CERO SERVIDORES / SEGURO)
160 // =====
161 document.getElementById('btn-descargar-backup').addEventListener('click', () => {
162     const panelGlobal = document.getElementById('panel-guardado-local');
163
164     panelGlobal.style.display = 'none';
165     const codigoLimpio = obtenerHTMLPurificado();
166     panelGlobal.style.display = 'flex';
167
168     // Generar y forzar la descarga nativa del archivo limpio mediante el navegador
169     const blob = new Blob([codigoLimpio], { type: "text/html" });
170     const enlace = document.createElement("a");
171     enlace.href = URL.createObjectURL(blob);
172     enlace.download = nombreArchivo; // Conserva el nombre real (ej: index.html)
173
174     document.body.appendChild(enlace);
175     enlace.click();
176     document.body.removeChild(enlace);
177 });
178 }
179 // TRUCO UNIVERSAL PARA ENLACES LARGOS (Solo se activa en ?modo=editar)
180 if (modoEditarActivo) {
181     document.querySelectorAll('a').forEach(enlace => {
182         // Escuchamos el doble clic en cualquier enlace del portal
183         enlace.addEventListener('dblclick', (evento) => {
184             // Evitamos que el navegador abra el enlace original al hacer doble clic

```

```

185         evento.preventDefault();
186
187         // Capturamos la ruta actual para mostrársela al usuario como referencia
188         const rutaActual = enlace.getAttribute('href') || 'index.html';
189
190         // Abrimos la ventana flotante nativa del sistema
191         let nuevaRuta = prompt(
192             `⚙️ Configurar enlace para: "${enlace.innerText.trim()}"\n\nIngrese el
193             nombre del archivo de destino (ej: atencion.html o login.html):`,
194             rutaActual
195         );
196
197         // Si el usuario no canceló y escribió algo, actualizamos el href tras
198         // bastidores
199         if (nuevaRuta !== null && nuevaRuta.trim() !== "") {
200             enlace.setAttribute('href', nuevaRuta.trim());
201             alert(`✅ Enlace actualizado internamente a: ${nuevaRuta.trim()}`);
202         }
203     });
204 }
205 // TRUCO EXPANDIDO: CONFIGURACIÓN DE BOTONES Y CAJAS DE BÚSQUEDA (?modo=editar)
206 if (modoEditarActivo) {
207     // A) Asegurar que todos los botones del navbar sean editables al hacer un clic
208     document.querySelectorAll('button, .btn, [class*="btn"]').forEach(boton => {
209         boton.setAttribute('contenteditable', 'true');
210     });
211
212     // B) Interceptar el doble clic en las Cajas de Búsqueda (Inputs)
213     document.querySelectorAll('input[type="text"], input[type="search"]').forEach(caja => {
214         caja.addEventListener('dblclick', (evento) => {
215             evento.preventDefault();
216
217             const textoSugerenciaActual = caja.getAttribute('placeholder') || '';
218
219             // Ventana flotante nativa para cambiar el texto de sugerencia interno
220             let nuevoPlaceholder = prompt(
221                 `⚙️ Configurar texto de sugerencia para la barra de búsqueda:\n\nIngrese
222                 el nuevo texto que verá el ciudadano:`,
223                 textoSugerenciaActual
224             );
225
226             if (nuevoPlaceholder !== null) {
227                 caja.setAttribute('placeholder', nuevoPlaceholder.trim());
228                 alert(`✅ Barra de búsqueda actualizada visualmente.`);
229             }
230         });
231     });
232 }

```